

TrackMaster Part-2

An Introductory Session for Beginners, Introducing them to the
Basics of Electronics and Microcontroller Programming

Managed by:

Robotics and Automation Society IEEE

Ibraheem Saboor	ChairPerson
Muhammad Zain	Co-Chair
Fatima Mubashar	Co-Chair

Supervised by: Sir Rehan Naeem



Department of Electrical Engineering
University of Engineering and Technology Lahore

December 2025

Contents

Abbreviations	iii
1 Overview	1
2 Introduction	2
2.1 Pull-up and Pull-down Resistors	2
2.2 Switch Debouncing	3
2.3 Polling	3
3 Tutorial - The Serial Monitor	4
3.1 Objective	4
3.2 Requirements	4
3.3 Code	4
3.4 Observation	4
4 Experiment-1 : Reading Digital Input Signals	6
4.1 Explanation	6
4.2 Objective	6
4.3 Components Required	6
4.4 Circuit Patching	6
4.5 Code	7
4.6 Expected Output	7
4.7 Task-1	8
4.8 Task-2	8
5 Experiment-2 : Reading Analog Input Signals	9
5.1 Explanation	9
5.2 Objective	9
5.3 Components Required	9
5.4 Circuit Patching	10
5.5 Code	10
5.6 Expected Output	10
5.7 Task-1	10
6 Experiment-3 : Interfacing an IR Sensor	12
6.1 Explanation	12
6.2 Objective	12
6.3 Components Required	12

6.4	Circuit Patching	12
6.5	Code	13
6.6	Expected Output	13
6.7	Task-1	13

Abbreviations

MCU	M icro C ontroller U nit
GPIO	G eneral P urpose I nput O utput
IR	I nfra R ed
LED	L ight E mitting D iode
ADC	A nalog to D igital C onverter

Chapter 1

Overview

This lab aims to provide anyone without any prior experience in electronics with a starting point to begin their journey. We will learn what a microcontroller is, how it works, what are its applications and what are its limitations. For this particular series of workshops, we will be using the Arduino Nano, an inexpensive entry level microcontroller development board equipped with all the functionality and tools that you might need for nearly every application at this level. This is the first part of a 3-part workshop, at the end of which, you will have made your own Line Following Robot. All the skills required to bring this project to completion will be taught during these 3 workshops. Each workshop will have its own set of objectives and goals:

- The first workshop will teach you basic MCU programming. You will learn to control outputs on GPIO pins and generate as well as control PWM signals.
- The second workshop will teach you input detection as well as active and passive polling. You will learn how to deal with noise and both level and edge triggered inputs.
- The third workshop will teach you how to assemble your own hardware and deal with all the problems that come with assembling physical circuits.

Chapter 2

Introduction

The MCU operates in a binary world (1s and 0s), which are distinguished on the basis of the voltage threshold. Whereas the physical world is analog, consisting of continuous variations rather than a simple voltage level. To bridge these two, we detect physical changes and convert them into electrical signals via sensors and transducers, which are then processed by the MCU. Usually, the voltage level of V_{CC} is assigned to digital value 1 and Ground voltage level to a digital value of 0. This data exchange is performed through the GPIO pins of the MCU. Thus, when we read digital input on the GPIO pins, the MCU becomes aware of the voltage level on all the GPIO pins and processes the data easily.

2.1 Pull-up and Pull-down Resistors

Whenever we want to read a signal as digital input, the circuitry inside the MCU will decide whether the voltage level being read is to be mapped to 0 or 1. This wouldn't pose an issue if we only had either V_{CC} or GND in the physical world, but the physical world is analog which means that we might have some voltage between 0 and 1 (say 2V, where $V_{CC}=5V$ and $GND=0V$). So, whenever we design a circuit where we want a digital input, we always make sure no other voltage levels are possible. The following 2 configurations are commonly used, depending on whether we want an active-high or active-low signal:

- **Pull-up Configuration :** A resistor connects the pin reading input to V_{CC} . And the pin is connected to G_{ND} through a switch. The default state (when the switch is open) is HIGH. When the switch is closed (connected to G_{ND}), the current flows to G_{ND} , pulling the pin LOW since the connection to G_{ND} (direct) is stronger than the connection to V_{CC} (through a resistor) .
- **Pull-down Configuration :** A resistor connects the pin reading input to G_{ND} . And the pin is connected to V_{CC} through a switch. The default state (when the switch is open) is LOW. When the switch is closed (connected to V_{CC}), the

current flows to V_{CC} , pulling the pin HIGH since the connection to G_{ND} (through a resistor) is weaker than the connection to V_{CC} (direct) .

When MCU's pin is configured as input and nothing is connected to it, the pin is in a "floating" state, it acts as an antenna and picks noise causing the MCU to read random 0s and 1s. This is undesirable if we want reliability.

2.2 Switch Debouncing

In the real world, mechanical switches do not close/open instantaneously, they usually vibrate and this creates a rapid series of on/off signals that the MCU might interpret as multiple button presses.

- **Debouncing** is the process of filtering out these unwanted multiple presses, either via hardware (capacitors) or software (ignoring changes for a few milliseconds after the first detection, e.g. using `delay()`).

To implement debouncing, we will be using the software fix. Debouncing is usually needed when we are using edge detection, for level detection we may skip debouncing.

2.3 Polling

How does the MCU know when a button was pressed? The answer is simple, it "asks" the pin.

- **Active Polling** : The CPU continuously checks the status of an I/O pin within a loop (e.g., asking, 'Is the button pressed?'). This depends on the clock speed of the MCU. This implementation is easy to implement but not very efficient in terms of CPU usage.
- **Passive Polling** : This type of polling utilized the interrupt timer built into the processor. This implementation is less CPU exhaustive and more complex to implement.

For now, passive polling is outside the scope of our workshop, so we will be using active polling.

The solutions to all the tasks can be found in the [Solutions Drive](#).

Chapter 3

Tutorial - The Serial Monitor

The Serial Monitor is a built-in tool in the Arduino IDE that acts as a communication window between your computer and the Arduino board. It shows the data on the monitor which makes it extremely useful for debugging.

3.1 Objective

Configure the serial interface to allow the MCU to send text data to the computer which we will use for debugging.

3.2 Requirements

- You must have the Arduino IDE and all the relevant drivers installed.
- You need to be familiar with the code structure.
- You need to be able to upload code to an Arduino using the IDE.

3.3 Code

```
1 void setup() {  
2     Serial.begin(9600);  
3 }  
4  
5 void loop() {  
6     Serial.println("Working");  
7 }  
8
```

3.4 Observation

Upload the code and open the Serial Monitor window on your IDE. To do so, in the top right corner of the windows, click Serial Monitor.



FIGURE 3.1

A window will appear in the bottom. In the top right corner of the window, click on baud drop down menu and select baud rate to 9600.

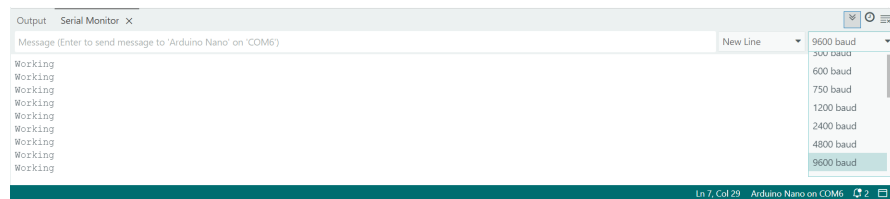


FIGURE 3.2

Now you will be able to see the string you entered in `Serial.println()`, printing in the serial monitor.

Chapter 4

Experiment-1 : Reading Digital Input Signals

4.1 Explanation

To interact with any physical environment, we need to be able to read inputs and convert the data into a form which the MCU can understand. This data may be from a smoke alarm detecting smoke, from the pressure sensor in a tyre indicating low pressure or even a doorbell being rung by someone. In our case, we will use a simple push switch as our input source.

4.2 Objective

To configure a GPIO pin as a digital input for reading the state of a tactile push button and to implement two distinct control logics.

4.3 Components Required

- Arduino Nano
- Breadboard
- Push button
- $10k\Omega$ resistor
- Jumper wires

4.4 Circuit Patching

Configuration: Pull-down Resistor (Active High)

- Place Arduino Nano and push button at appropriate location on the breadboard. (Make sure which terminals of the push switch are not shorted and place them into the circuit accordingly).

- Connect one terminal of the push button to the 5V pin of the Arduino Nano.
- Connect the other terminal of the push button to Digital Pin D5.
- From the same terminal connected to D5, insert a $10k\Omega$ resistor connecting to GND.

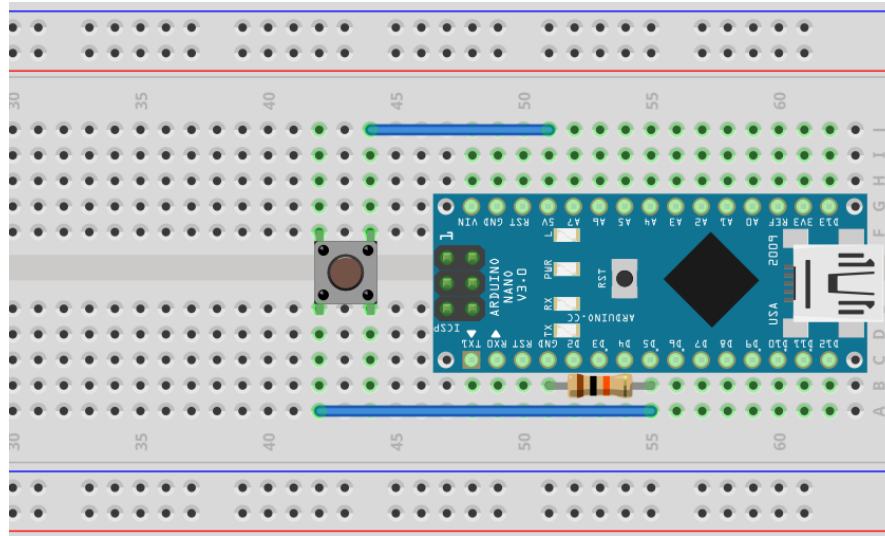


FIGURE 4.1

4.5 Code

```

1  int button = 5;
2
3  void setup() {
4      pinMode(button, INPUT);
5      Serial.begin(9600);
6  }
7  // digitalRead( Pin ) is a command which outputs 0 when
8  // it detects GND on the pin and 1 when it detects Vcc on the pin
9  void loop() {
10     if (digitalRead(button) == 1){
11         Serial.println("Button pressed");
12     }
13     else{
14         Serial.println("Button released");
15     }
16 }

```

4.6 Expected Output

The expected output should look like Figure 4.2 in the in the serial monitor.

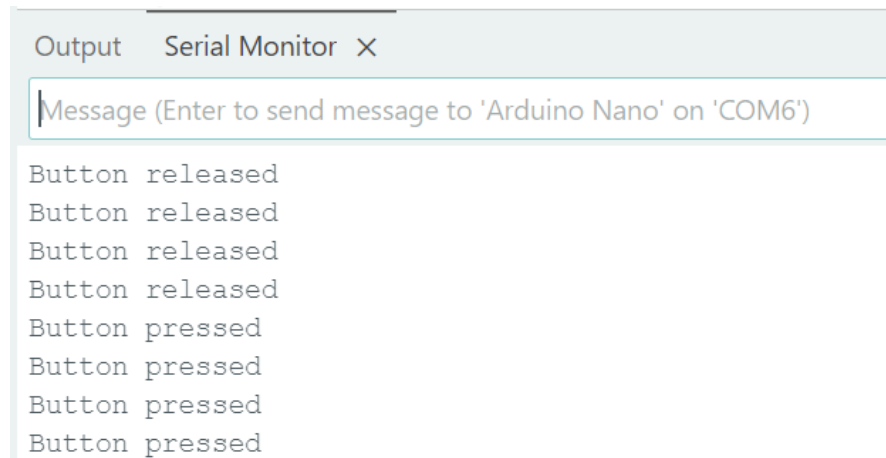


FIGURE 4.2

"Button pressed" when you press the push button and "Button released" when the button is no longer pressed.

4.7 Task-1

Goal: The Onboard LED should remain ON only while the button is physically held down while printing the status as well.

- Read the digital pin connected to the button.
- If the reading is Active (e.g., LOW for pull-up configuration), write HIGH to the LED pin.
- Else, write LOW to the LED pin.
- Print "Button Pressed" or "Button Released" to Serial Monitor.

4.8 Task-2

Goal: Pressing and releasing the button once should toggle the LED (turn it ON if OFF, and vice versa).

- We need to detect the edge (the transition from High to Low), not just the level.
- Store the previousState of the button.
- If currentState is Pressed AND previousState was Released: Toggle the LED variable (`ledState = !ledStatePrev`). Update the physical LED.
- Implement a small delay (e.g., 50ms) for software debouncing to prevent flickering.

Chapter 5

Experiment-2 : Reading Analog Input Signals

5.1 Explanation

Reading digital inputs is very useful as it is the basic of real-world interaction for all digital circuits. But there are also many cases where we need to read Analog inputs. For the digital inputs, the value is mapped to either 0 or 1. For the analog signals, the range of values that we can map to depends on the resolution of the ADC on the MCU. Suppose, we have a 10-bit ADC (as is the case for our arduino Nano). An input of 3.825V would be translated to a digital value of (793). An easy way to theoretically calculate the ADC value is to use the formula:

$$ADCValue = AnalogVoltage * (2^{Resolution} / V_{cc})$$

5.2 Objective

Read analog voltage using an Analog pin, process the data and display it on the Serial Monitor.

5.3 Components Required

- Arduino Nano
- Breadboard
- Potentiometer
- Resistors
- LEDs
- Jumper wires

5.4 Circuit Patching

- Connect the two outer terminals of the potentiometer to the 5V and GND pins respectively.
- Connect the center terminal of the potentiometer to Analog Pin A0.

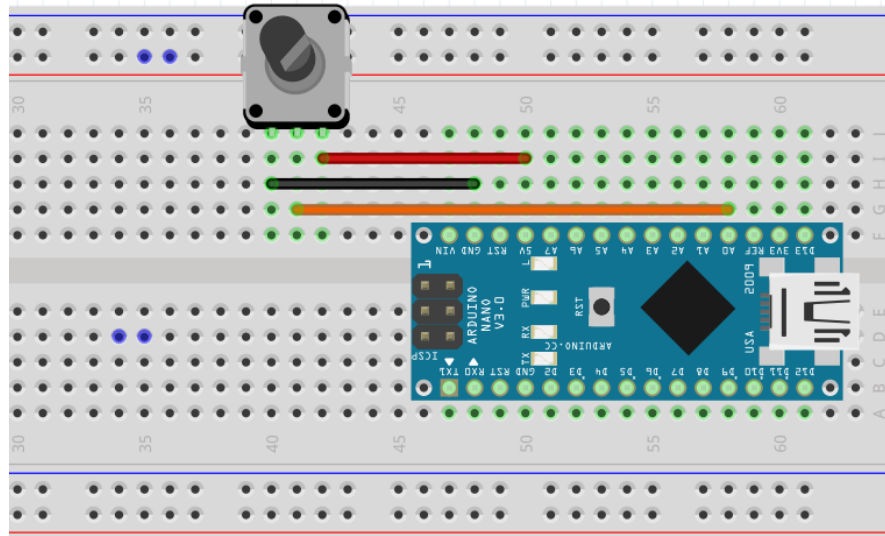


FIGURE 5.1

5.5 Code

```

1  int analogPin = A0;
2
3  void setup() {
4      pinMode(analogPin, INPUT);
5      Serial.begin(9600);
6  }
7  // analogRead is a command which returns a value between 0 and 1023,
8  //where 0 corresponds to 0V and 1023 corresponds to 5V and all the
9  //values in between are mapped linearly.
10 void loop() {
11     Serial.println(analogRead(analogPin));
12 }

```

5.6 Expected Output

The end result should look like READING_ANALOG_SIGNAL.mp4 in solution drive

5.7 Task-1

Goal: Control the brightness of an external LED based on the potentiometer position.

- Perform `analogRead()` on the potentiometer pin.

- Print the raw value to the Serial Monitor to observe the range. Map the value: The ADC input range (e.g., 0-1023) might not match the PWM output range (e.g., 0-255). Use a mapping function/calculation.
- Use `analogWrite()` (PWM) on the LED pin using the mapped value.

Chapter 6

Experiment-3 : Interfacing an IR Sensor

6.1 Explanation

An IR Sensor is a device which uses Infrared Light which we cannot see with the naked eye. It is commonly used for object detection, distance measurement, motion detection, and remote control systems. A typical IR module has 2 LEDs (an IR transmitter LED and a photodiode receiver). The transmitter sends out IR rays which may bounce off an object and get reflected into the receiver. In this way it detects whether something is placed in front of it.

6.2 Objective

To configure the IR sensor as a digital input device and observe its output on the Serial Monitor.

6.3 Components Required

- Arduino Nano
- Breadboard
- IR Sensor
- Jumper Wires

6.4 Circuit Patching

- Connect the VCC pin of the IR Sensor to the 5V pin on the Arduino Nano.
- Connect the GND pin of the IR Sensor to the GND pin on the Arduino Nano.
- Connect the OUT (or Signal) pin of the IR Sensor to Digital Pin D2.

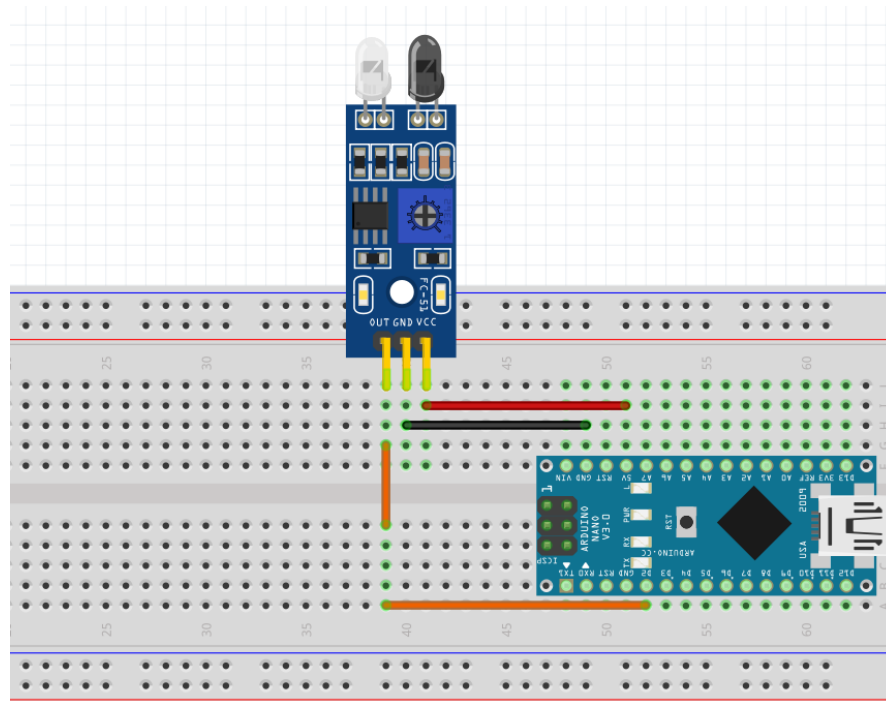


FIGURE 6.1

6.5 Code

```

1  int irPin = 2;
2
3  void setup() {
4      pinMode(irPin, INPUT);
5      Serial.begin(9600);
6  }
7
8  void loop() {
9      if (digitalRead(irPin) == 0){
10         Serial.println("Object Detected");
11     }
12     else{
13         Serial.println("Looking for Object");
14     }
15 }

```

6.6 Expected Output

The end result should look like OBJECT_DETECTION.mp4 in solution drive

6.7 Task-1

Goal: Turn on the Onboard LED when a light or reflective object passes the sensor.

- Read the pin status.
- **Calibration:** Most IR modules output HIGH when no reflection is detected (Dark/Void) and LOW when reflection is detected (Light object).
- If the sensor reads "Light/Reflection Detected" (LOW):
Turn Onboard LED ON.
Print "Object Detected" to Serial Monitor.
- Else:
Turn Onboard LED OFF.